Experiment 8

Aim

Write a program to implement Pulse Amplitude Modulation (PAM), Pulse Width Modulation (PWM) and Pulse Position Modulation (PPM) in Octave. For a given message signal, modulate a pulse train with PAM, PWM and PPM and plot the modulated signals.

Software Required

Octave (version 9.4.0)

Theory

1. Pulse Amplitude Modulation (PAM)

Pulse Amplitude Modulation (PAM) is a technique where the amplitude of equally spaced pulses varies according to the message signal while pulse width and position remain constant. It is simple to implement and widely used in communication systems, but it is more susceptible to noise since information is carried in amplitude.

2. Pulse Width Modulation (PWM)

Pulse Width Modulation (PWM) varies the width (duty cycle) of pulses in accordance with the message signal while keeping amplitude constant. It is more robust against noise compared to PAM and is commonly used in power control applications, motor drives, and signal processing systems due to its efficiency.

3. Pulse Position Modulation (PPM)

Pulse Position Modulation (PPM) encodes information by varying the position of pulses relative to a reference time while keeping amplitude and width constant. It provides better noise immunity than PAM and PWM, but requires precise synchronization between transmitter and receiver, making implementation slightly more complex.

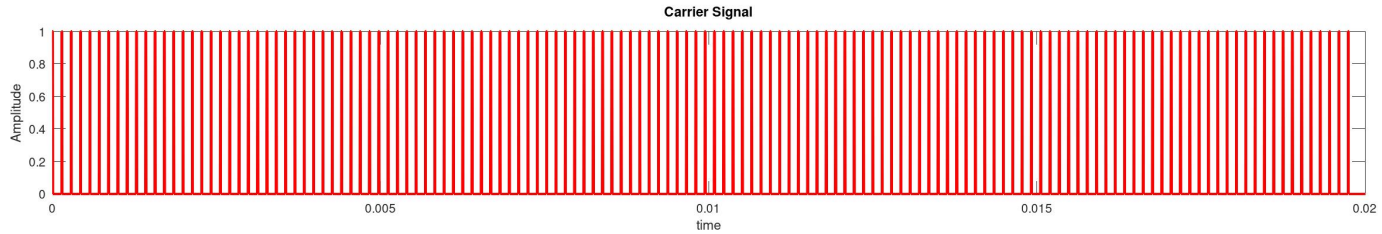
Code

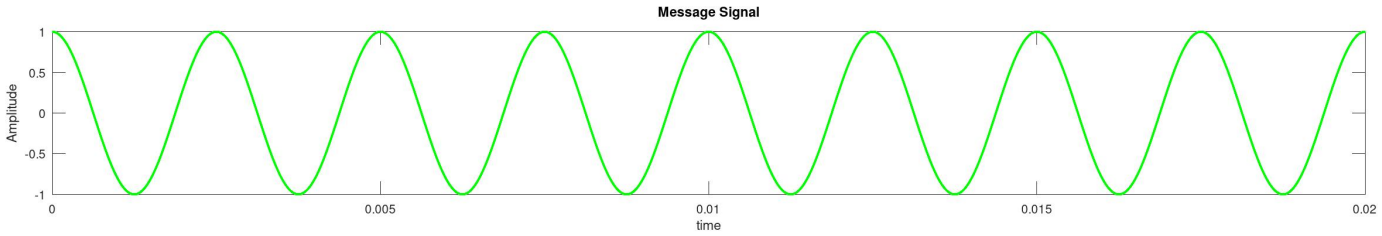
```
clc;
clear;

pkg load signal

tend = 0.02;
fs = 50000*10;
t = 0:1/fs:tend;
N = length(t);
dt = 1/fs;

##Carrier
fc = 7000;
tc = 1/fc;
s = int32(tc/dt)
c = zeros(1,N);
duty = 0.1;
n = tend/tc;
```

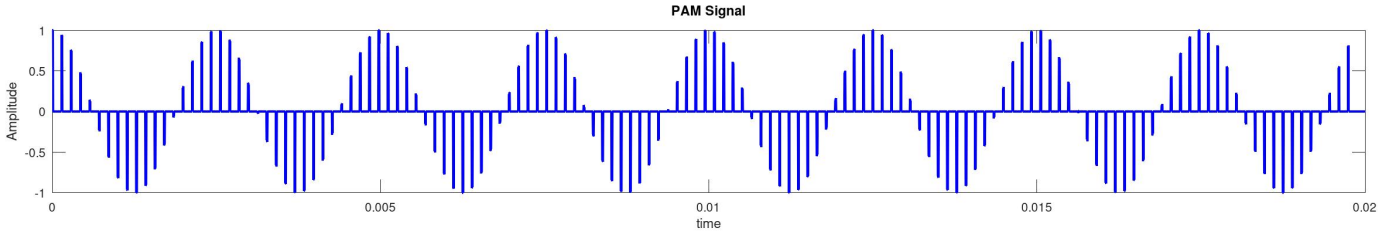
```
pulse_width = floor(0.1 * s);

for i = 1:n;
  c((i-1)*s+1:(i-1+duty)*s) = 1;
end

## Message Siganl
fm = 400;
m = cos(2*pi*fm*t);

## PAM Signal
pam = m.*c;

## PWM Signal
pwm = zeros(1,N);
for i = 1:n;
  duty = (m(int32(i*s))+1)/2;
  pwm((i-1)*s+1:(i-1+duty)*s) = 1;
end

ppm = zeros(1,N);
for i = 1:n;
  del = ((m(int32(i*s))+1)/2);
  start_idx = (i-1)*s + 1;
  shift = floor(del * (s - pulse_width));
  ppm(start_idx + shift:start_idx + shift + pulse_width - 1 )= 1;
end

set(0,"defaultlinelinewidth",2);
figure();
subplot(3,1,1);
plot(t,c,'r');
title("Carrier Signal");
xlabel("time");
ylabel("Amplitude");

subplot(3,1,2);
plot(t,m,'g');
title("Message Signal");
xlabel("time");
ylabel("Amplitude");

subplot(3,1,3);
plot(t,pam,'b');
title("PAM Signal");
xlabel("time");
ylabel("Amplitude");

figure();
```

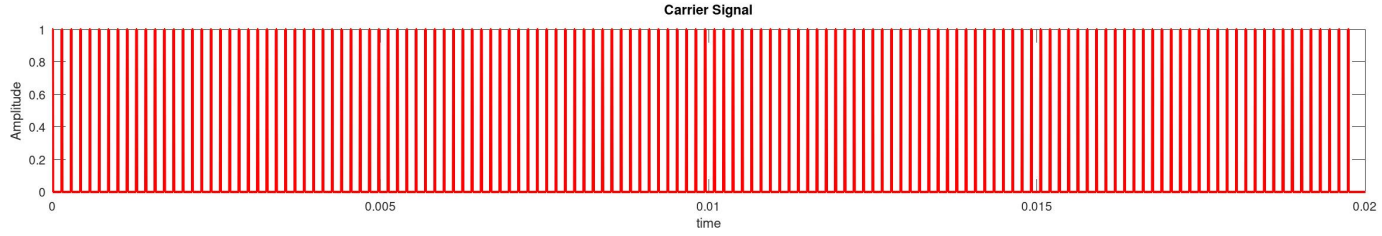
```
subplot(3,1,1);
plot(t,c,'r');
title("Carrier Signal");
xlabel("time");
ylabel("Amplitude");

subplot(3,1,2);
plot(t,pwm,'b');
title("PWM Signal");
xlabel("time");
ylabel("Amplitude");

subplot(3,1,3);
plot(t,m,'g');
title("Message Signal");
xlabel("time");
ylabel("Amplitude");

figure();
subplot(3,1,1);
plot(t,c,'r');
title("Carrier Signal");
xlabel("time");
ylabel("Amplitude");

subplot(3,1,2);
plot(t,ppm,'b');
title("PPM Signal");
xlabel("time");
ylabel("Amplitude");

subplot(3,1,3);
plot(t,m,'g');
title("Message Signal");
xlabel("time");
ylabel("Amplitude");
```

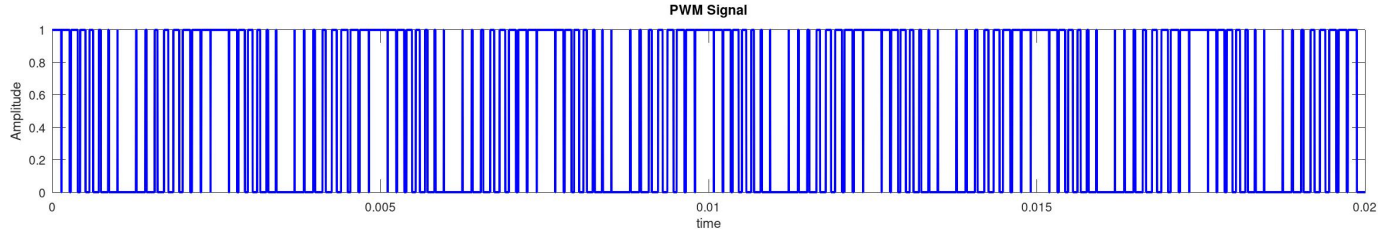
Simulation Results and Plots

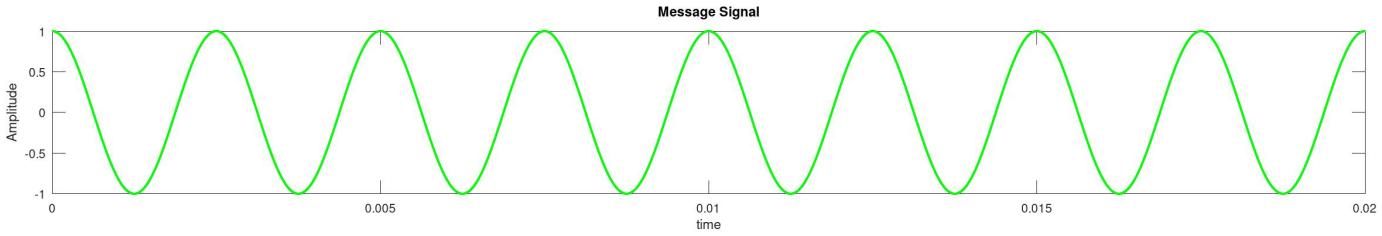


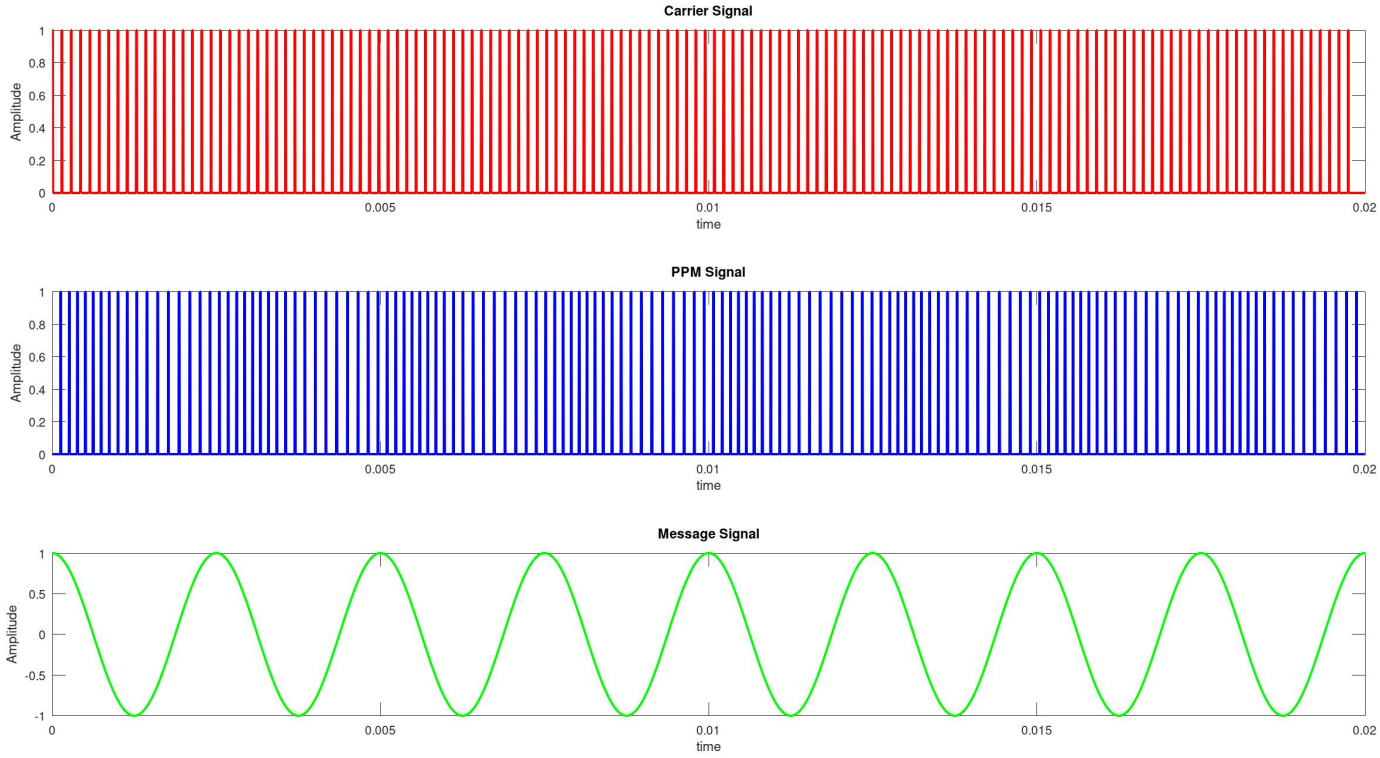












Conclusion

PAM was successfully implemented and analyzed using Octave. The modulated signal showed amplitude variations corresponding to the message signal. Although simple and efficient, PAM is sensitive to noise, which affects signal quality. The experiment helped understand modulation, demodulation, and filtering processes in digital communication systems. PWM modulation was successfully generated and demodulated. The pulse width varied according to the message signal, and the original signal was reconstructed using averaging and thresholding. PWM demonstrated better noise immunity and efficiency compared to PAM, making it suitable for practical applications like power electronics and communication systems. PPM was implemented successfully, showing pulse position variation based on the message signal. The demodulation process recovered the original data accurately. PPM provides strong noise immunity but requires synchronization. This experiment enhanced understanding of timing- based modulation techniques and their importance in modern communication systems.

Experiment 9

Aim

Write a program to implement Pulse Code Modulation (PCM). Sample an analog message signal, quantize it, encode the samples and generate a bit stream. Reconstruct the original message signal back at the receiver side. Plot the reconstructed signal for low number of bits per sample and higher number of bits per sample.

Software Required

Octave (version 9.4.0)

Theory

Pulse Code Modulation (PCM)

PCM is a technique used to convert an analog signal into a digital signal for transmission. Steps in PCM:

1. Sampling The analog signal is sampled at regular intervals. According to the Nyquist Sampling Theorem, the sampling frequency must be at least twice the maximum frequency of the signal.
2. Quantization Sampled values are rounded to the nearest discrete levels. Number of levels = 2^n , where n is bits per sample. This introduces quantization error.
3. Encoding Each quantized value is converted into a binary code, forming a bit stream. At Receiver: Bit stream is decoded Signal is passed through a reconstruction filter Original signal is approximately recovered

Code

```
clc;
clear;
close all;


fs = 1000;
t = 0:1/fs:1;
N = length(t);


fm = 5;
m = cos(2*pi*fm*t);

function [mq, q_index, bitstream, m_rec] = pcm_process(m, bits)

    L = 2^bits;


    m_min = min(m);
    m_max = max(m);
    m_norm = (m - m_min) / (m_max - m_min);


    q_index = floor(m_norm * (L-1));
```

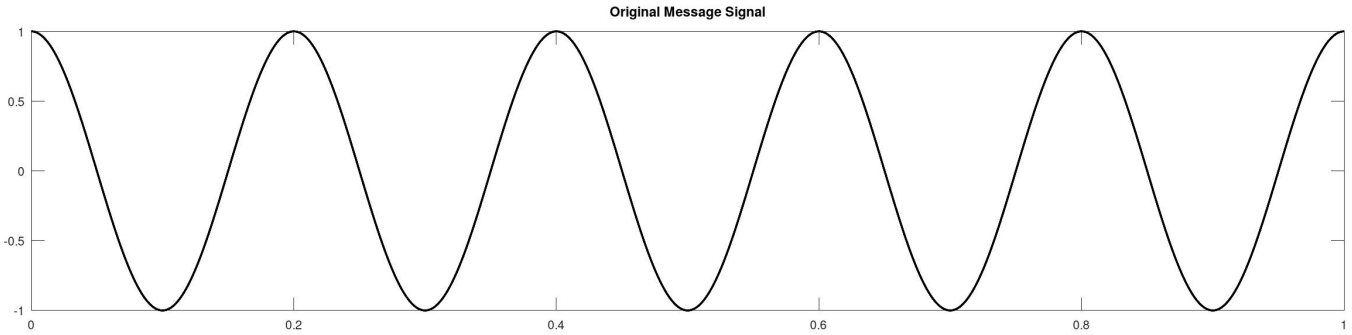
```
    mq = q_index / (L-1);


    bitstream = "";
    for i = 1:length(q_index)
        b = dec2bin(q_index(i), bits);
        bitstream = [bitstream b];
    end


    m_rec = mq * (m_max - m_min) + m_min;
end

%% LOW BIT
bits_low = 4;
[mq1, q1, bits1, rec1] = pcm_process(m, bits_low);

%% HIGH BIT
bits_high = 8;
[mq2, q2, bits2, rec2] = pcm_process(m, bits_high);



set(0,"defaultlinelinewidth",2);

figure();
subplot(2,1,1);
plot(t,m,'k');
title("Original Message Signal");

subplot(2,1,2);
stem(t,m,'filled');
title("Sampled Signal");


figure();
subplot(2,1,1);
stairs(t,mq1,'r');
title("Quantized Signal (4 bits)");

subplot(2,1,2);
stairs(t,mq2,'g');
title("Quantized Signal (8 bits)");

figure();
subplot(3,1,1);
plot(t,m,'k');
title("Original Signal");

```

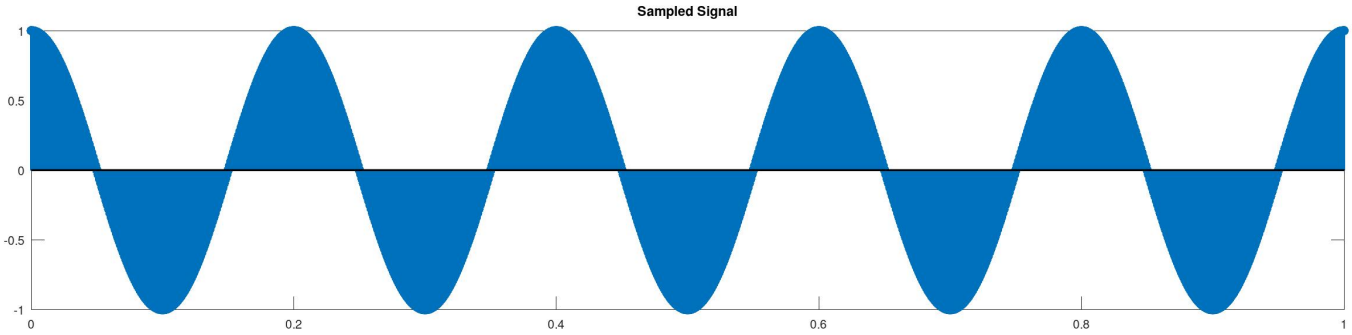
```
subplot(3,1,2);
plot(t,rec1,'r');
title("Reconstructed Signal (4 bits)");

subplot(3,1,3);
plot(t,rec2,'g');
title("Reconstructed Signal (8 bits)");


figure();
plot(t,m,'k'); hold on;
plot(t,rec1,'r--');
plot(t,rec2,'g');
legend("Original","4-bit PCM","8-bit PCM");
title("Comparison of Reconstruction");


disp("First 100 bits (4-bit PCM):");
disp(bits1(1:100));

disp("First 100 bits (8-bit PCM):");
disp(bits2(1:100));
```

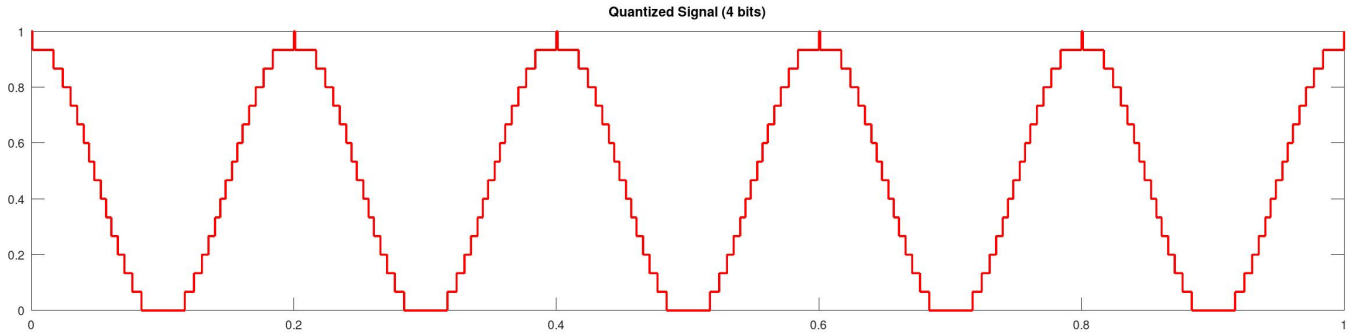
Output:

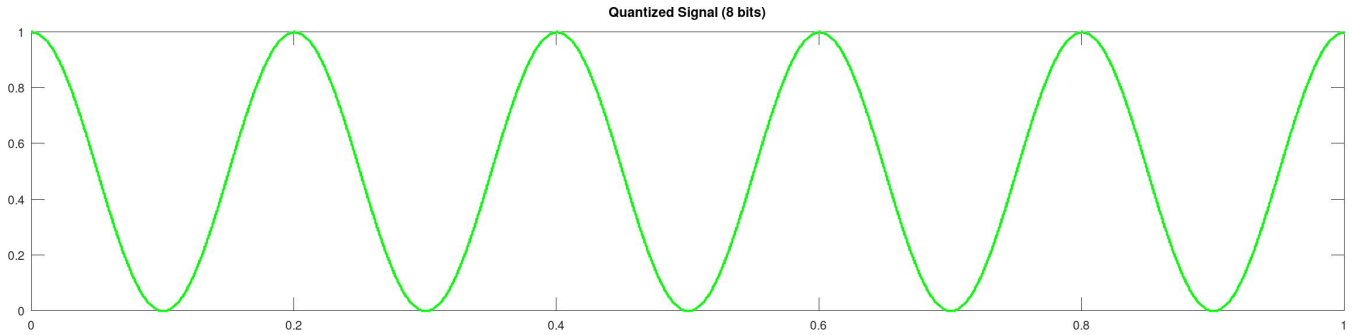
```
First 100 bits (4-bit PCM):
1111111011101110111011101110111011101110111011101110111011
1011101110111011101110111011101110111011101110111011101100
First 100 bits (8-bit PCM):
1111111111111110111111101111111011111101111111011111110111
1111001111101111111010111110011111100011110111111111
```

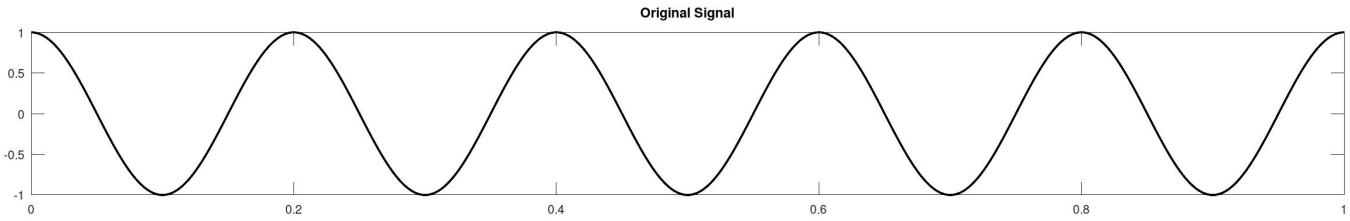
Simulation Results and Plots

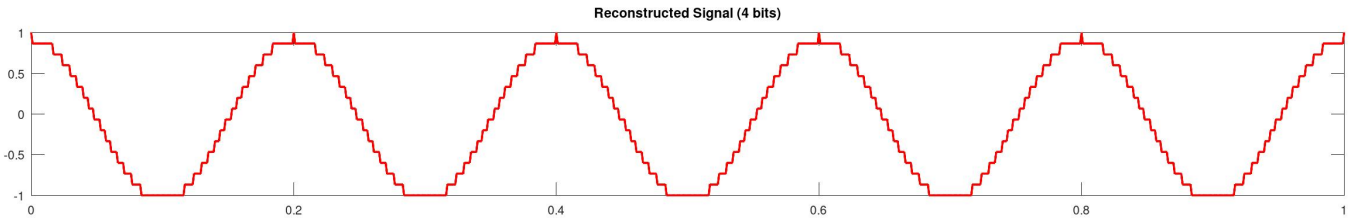


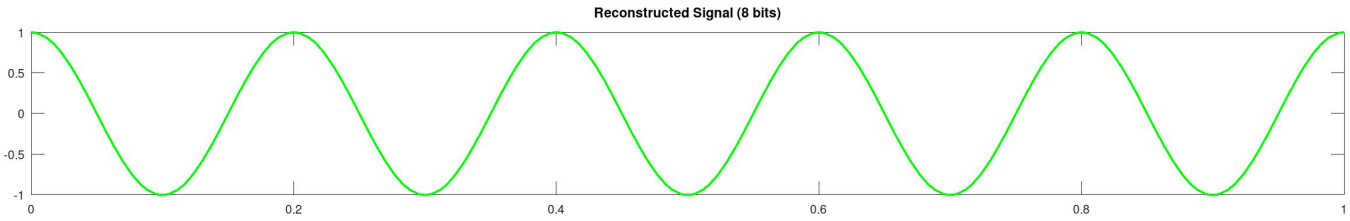


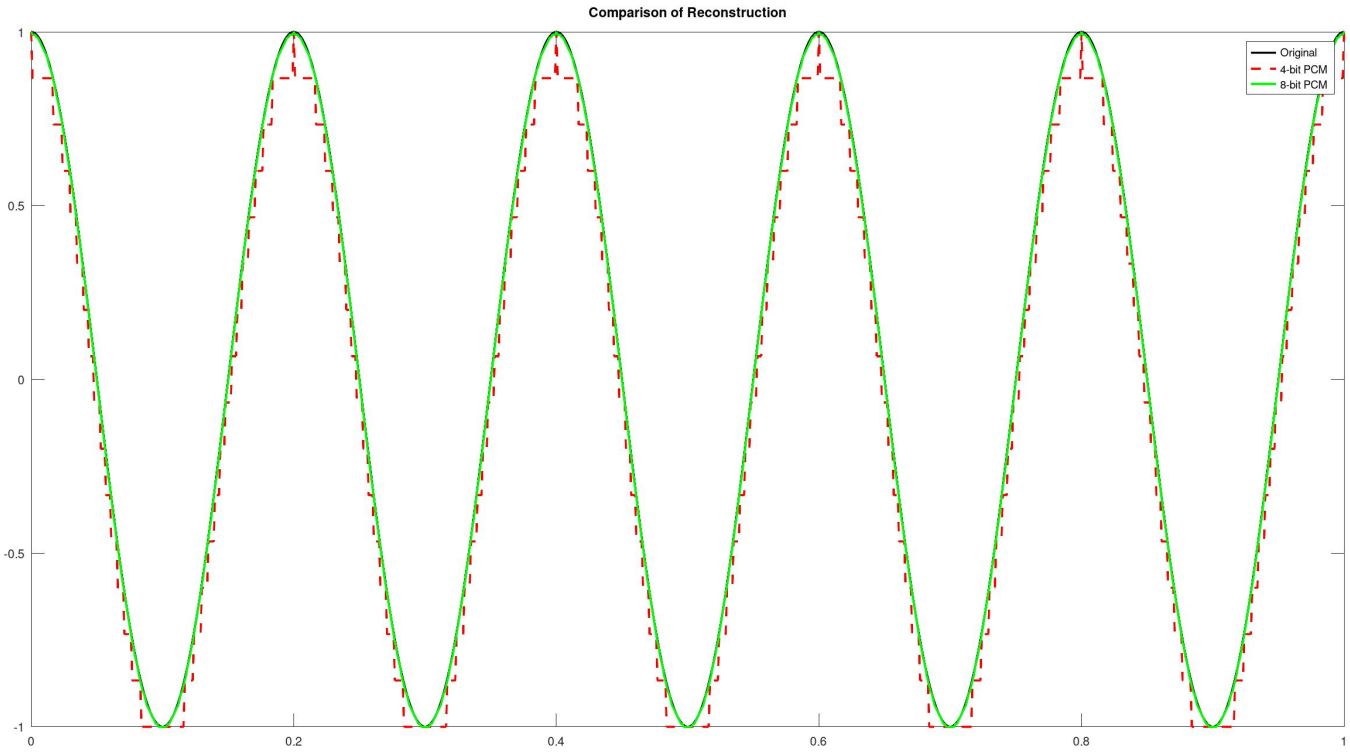





Original Signal
1
0.5
0
-0.5
-1
0
0.2
0.4
0.6
0.8
1


Reconstructed Signal (4 bits)
1
0.5
0
-0.5
-1
0
0.2
0.4
0.6
0.8
1


Reconstructed Signal (8 bits)
1
0.5
0
-0.5
-1
0
0.2
0.4
0.6
0.8
1


Comparison of Reconstruction
Original
4-bit PCM
8-bit PCM
1
0.5
0
-0.5
-1
0
0.2
0.4
0.6
0.8
1

Conclusion

The PCM system was successfully implemented and analyzed. It was observed that increasing the number of bits per sample improves signal reconstruction accuracy and reduces quantization noise. Lower bit levels produce noticeable distortion, while higher bit levels closely approximate the original signal, demonstrating the trade-off between bandwidth and signal quality.

Experiment 10

Aim

Write a program to implement Amplitude Shift Keying (ASK) and Frequency Shift Keying (FSK). Generate a random bit stream and modulate a carrier with ASK, and 2 carriers with different frequencies in the case of FSK. Use a non-coherent detector to demodulate both ASK and FSK modulated signals, reconstruct the original bit stream, and plot them. Observe the impact of noise on both the modulation techniques.

Software Required

Octave (version 9.4.0)

Theory

Amplitude Shift Keying (ASK)

ASK is a digital modulation technique in which the amplitude of a carrier signal is varied according to the binary data:
Binary 1 → High amplitude carrier
Binary 0 → Zero or low amplitude

- Demodulation (Non-coherent)
  Uses envelope detection (no phase synchronization needed)
  Detects amplitude variations to recover bits

- Effect of Noise
  Highly sensitive to noise (since noise affects amplitude)
  Performance degrades quickly in noisy channels

Shift Keying (FSK) – Brief Theory

FSK is a digital modulation technique where the frequency of the carrier signal changes according to the input bit:
Binary 1 Frequency f_1
Binary 0 Frequency f_2

- Demodulation (Non-coherent)
  Uses band-pass filters or energy detectors
  Detects which frequency is present to decide the bit

- Effect of Noise
  More robust than ASK
  Noise affects amplitude more than frequency, so FSK performs better in noisy environments

Code

- ASK

```
clc;
clear;

pkg load signal

tend = 0.01;
fs = 50000*10;
```

```
t = 0:1/fs:tend;
N = length(t);
dt = 1/fs;


fc = 5000;
c = cos(2*pi*fc*t);
n = 20;
mn=randi([0 1],1,n);
mt= zeros(1,N); #message signal
tb = N/n;
fm = 1/tb;
tbs = 0:tend/n:tend
##tb_index = (1/n)*[0:N-1]
##mt(1:100) = 1;

##a = mn(1)
##j = 1;
##for i = 1:N;
##  if (t(i)>tbs(j) && t(i)<tbs(j+1))
##    mt(i) = mn(j);
##    j = j+1;
##  end
##end


for i = 1:n;
  mt((i-1)*tb+1:i*tb) = mn(i);
end

ask = mt.*c;
noisy_ask = ask + sqrt(0.1)*randn(1,N);

set(0,"defaultlinelinewidth",2);
figure();
subplot(3,1,1);
plot(t,mt,'r');
title("Message Signal");
xlabel("time");
ylabel("Amplitude");

subplot(3,1,2);
plot(t,c,'g');
title("Carrier Signal");
xlabel("time");
ylabel("Amplitude");

subplot(3,1,3);
plot(t,noisy_ask);
```

```
title("ASK Signal");
xlabel("time");
ylabel("Amplitude");

##Demodulation
a = abs(noisy_ask);
figure();
##subplot(3,1,1);
##plot(t,a,'r');

coeff = fir1(200,(fm)/(fs/2));
fil = filter(coeff,1,a);
##subplot(3,1,2);
##plot(t,fil,'g');

half = (max(fil)+min(fil))/2
dm_sig = zeros(1,N);
for i = 1:N;
  if (fil(i)>half)
    dm_sig(i) = 1;
  elseif(fil(i)<half)
    dm_sig(i) = 0;
  end
end

subplot(3,1,1);
plot(t,mt,'r');
title("Original Signal");
xlabel("time");
ylabel("Amplitude");

subplot(3,1,2);
plot(t,dm_sig);
title("ASK Signal");
xlabel("time");
ylabel("Amplitude");


```

- FSK

```
clc;
clear;

pkg load signal

tend = 0.01;
fs = 50000*10;
t = 0:1/fs:tend;
```

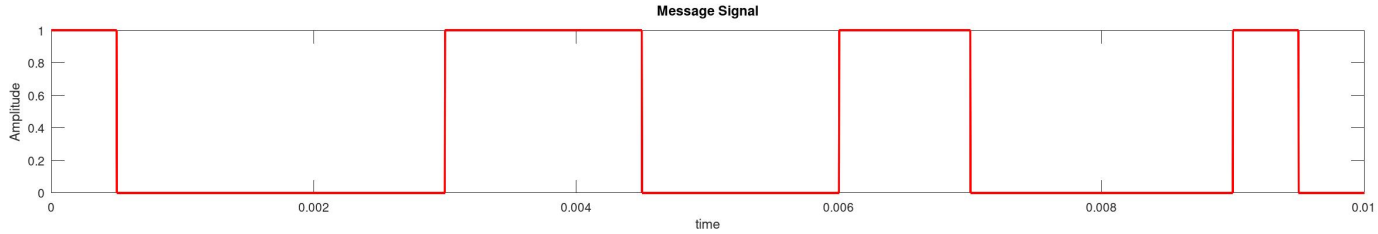
```
N = length(t);
dt = 1/fs;


fc1 = 5000;
c1 = cos(2*pi*fc1*t);
fc2 = 10000;
c2 = cos(2*pi*fc2*t);

n = 20;
mn=randi([0 1],1,n);
mt= zeros(1,N); #message signal
tb = N/n;
fm = 1/tb;
tbs = 0:tend/n:tend
##tb_index = (1/n)*[0:N-1]
##mt(1:100) = 1;

##a = mn(1)
##j = 1;
##for i = 1:N;
##  if (t(i)>tbs(j) && t(i)<tbs(j+1))
##    mt(i) = mn(j);
##    j = j+1;
##  end
##end


for i = 1:n;
  mt((i-1)*tb+1:i*tb) = mn(i);
end

fsk = zeros(1,N);
for i = 1:n;
  if(mn(i)==0)
    fsk((i-1)*tb+1:i*tb) = cos(2*pi*fc1*t((i-1)*tb+1:i*tb));
  elseif(mn(i)==1)
    fsk((i-1)*tb+1:i*tb) = cos(2*pi*fc2*t((i-1)*tb+1:i*tb));
   end
end


noisy_fsk = fsk + sqrt(0.1)*randn(1,N);
set(0,"defaultlinelinewidth",2);
figure();
subplot(2,1,1);
plot(t,mt,'r');
title("Message Signal");
xlabel("time");
```

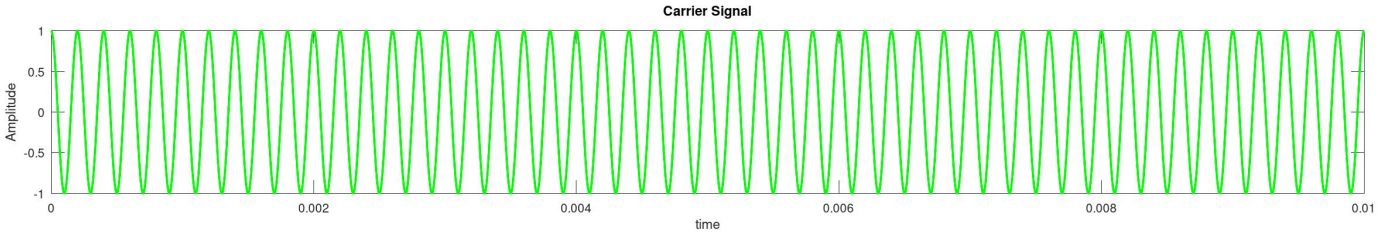
```
58  ylabel("Amplitude");
59
60  subplot(2,1,2);
61  plot(t,noisy_fsk);
62  title("FSK Signal (with noise)");
63  xlabel("time");
64  ylabel("Amplitude");
65
66  ##figure();
67  ##subplot(2,1,1);
68  ##plot(t,c1,'r');
69  ##title("Carrier Signal(Low Frequency)");
70  ##xlabel("time");
71  ##ylabel("Amplitude");
72  ##
73  ##subplot(2,1,2);
74  ##plot(t,c2,'g');
75  ##title("Carrier Signal(Low Frequency)");
76  ##xlabel("time");
77  ##ylabel("Amplitude");
78
79
80
81  ##Demodulation
82
83  figure();
84
85
86  coeff = fir1(200,(fc1)/(fs/2));
87  fil1 = filter(coeff,1,noisy_fsk);
88  subplot(3,1,1);
89  plot(t,fil1,'g');
90
91  ##filter ask signal
92  a = abs(fil1);
93  subplot(3,1,2);
94  plot(t,a,'r');
95
96  coeff1 = fir1(200,(fm)/(fs/2));
97  fil = filter(coeff1,1,a);
98  subplot(3,1,3);
99  plot(t,fil,'g');
100
101 half = (max(fil)+min(fil))/2
102 dm_sig = zeros(1,N);
103 for i = 1:N;
104   if (fil(i)>half)
105     dm_sig(i) = 0;
106   elseif(fil(i)<half)
```

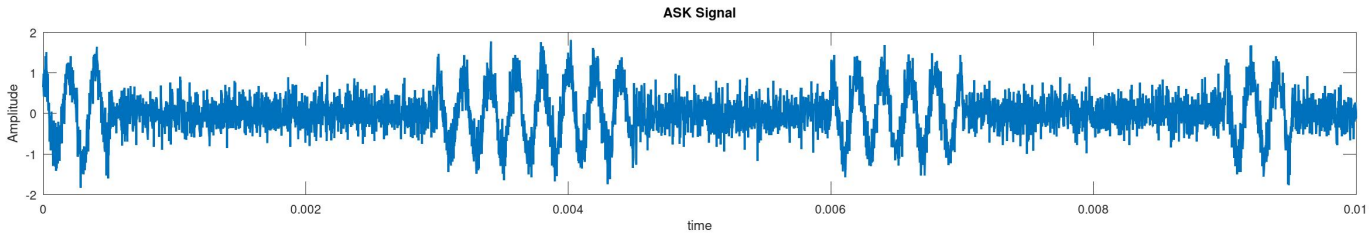
```
107       dm_sig(i) = 1;
108     end
109   end
110
111   figure();
112   subplot(2,1,1);
113   plot(t,mt,'r');
114   title("Original Signal");
115   xlabel("time");
116   ylabel("Amplitude");
117
118   subplot(2,1,2);
119   plot(t,dm_sig,'g');
120   title("Demodulated Signal");
121   xlabel("time");
122   ylabel("Amplitude");
123
```

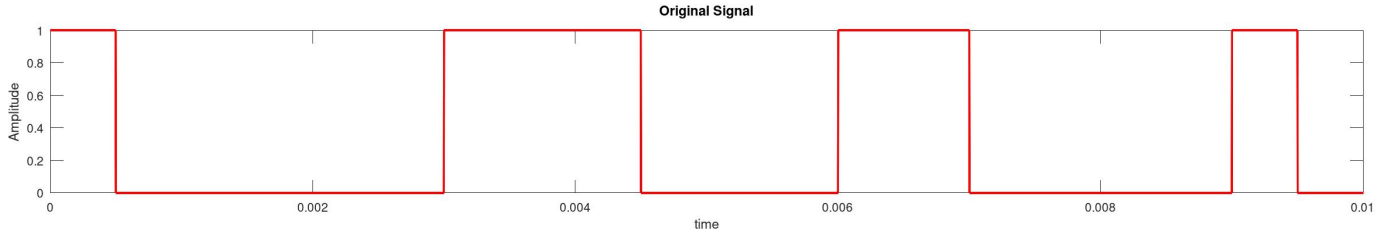
Simulation Results and Plots

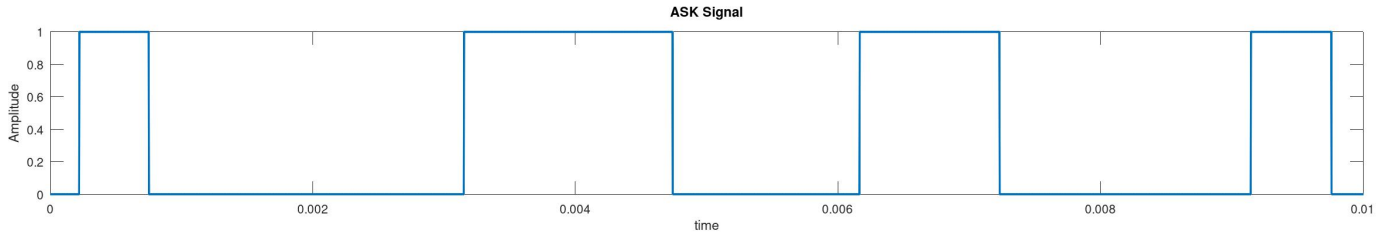
ASK



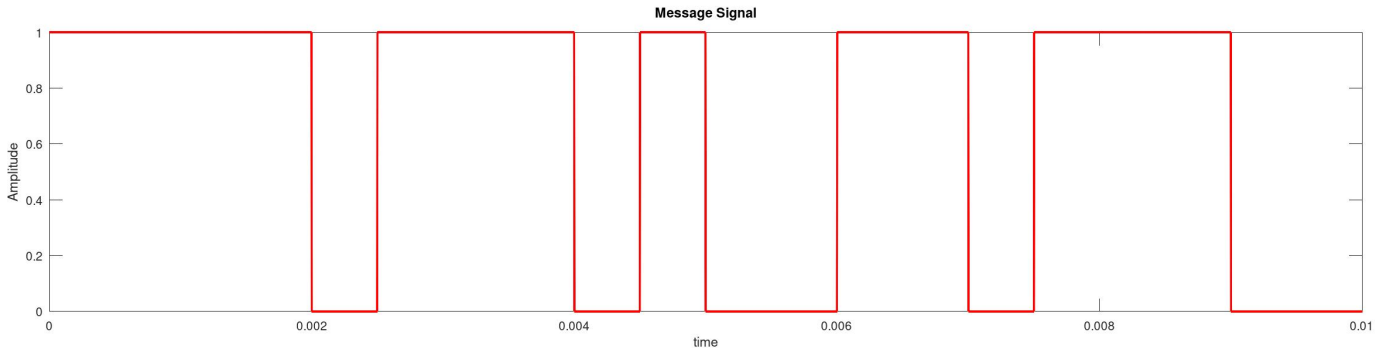


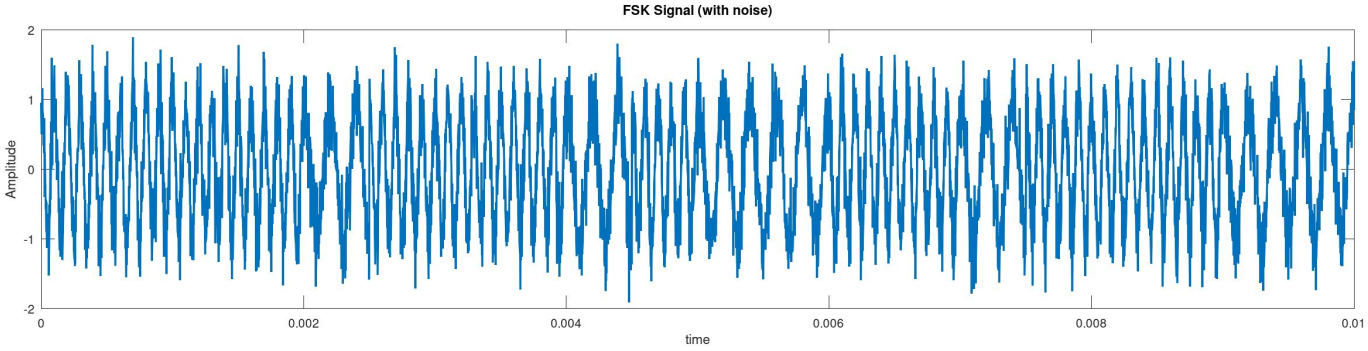


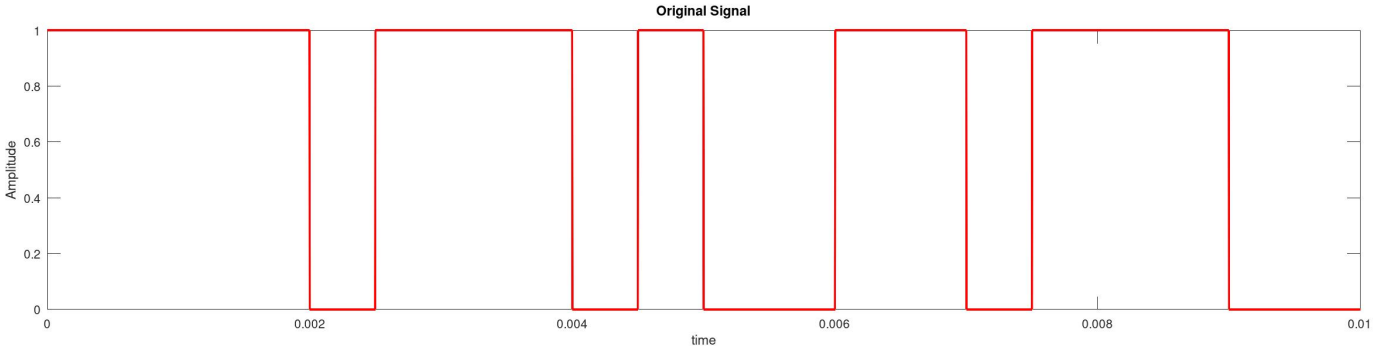


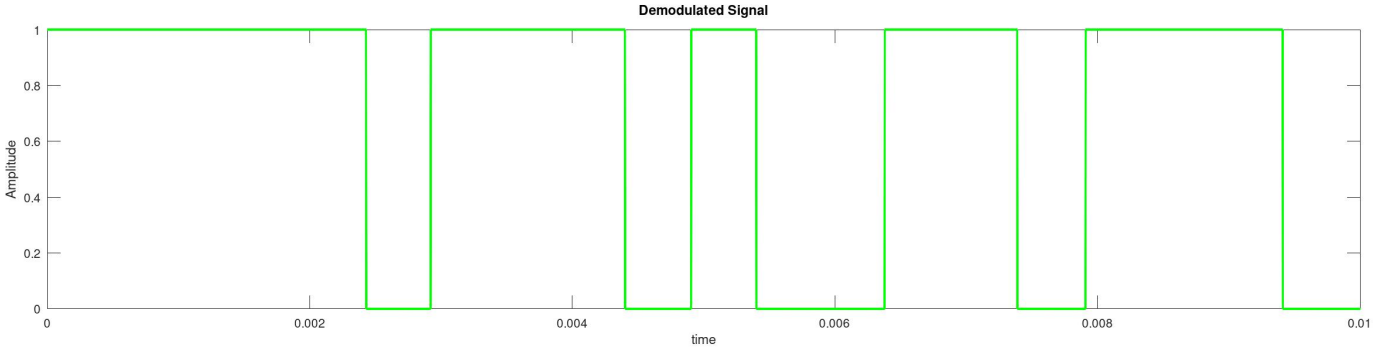


FSK


Message Signal
Amplitude
time
0
0.2
0.4
0.6
0.8
1
0.002
0.004
0.006
0.008
0.01


FSK Signal (with noise)
Amplitude
time
-2
-1
0
1
2
0.002
0.004
0.006
0.008
0.01


Original Signal
Amplitude
time
0
0.2
0.4
0.6
0.8
1
0.002
0.004
0.006
0.008
0.01


Demodulated Signal
Amplitude
time
0
0.2
0.4
0.6
0.8
1
0.002
0.004
0.006
0.008
0.01

Conclusion

The ASK and FSK modulation schemes were successfully implemented and analyzed. ASK showed higher susceptibility to noise due to amplitude variations, while FSK demonstrated better noise immunity. The reconstructed signals confirmed proper demodulation. This experiment highlights the trade-off between simplicity and robustness in digital communication systems under noisy conditions.

Experiment 11

Aim

Write a program to implement Binary Phase Shift Keying (BPSK). Generate a random bit stream and modulate a carrier with BPSK. Use a coherent detector to demodulate the signal and recover the original bit stream. Plot all the signals and observe the effects with noise.

Software Required

Octave (version 9.4.0)

Theory

Binary Phase Shift Keying (BPSK)

Binary Phase Shift Keying (BPSK) is a digital modulation technique in which the phase of a carrier signal is changed according to the input bit:
Bit 1 $\rightarrow$ 0˚ phase Bit 0 $\rightarrow$ 180˚ phase

Modulation

The transmitted signal is:
$s(t) = b(t)cos(2\pi fct)$
where $b(t) = +1$ for bit 1 and -1 for bit 0.

Demodulation (Coherent Detection)

Receiver uses a synchronized carrier signal
Multiplies received signal with carrier
Integrates over bit duration
Positive $\rightarrow$ 1 Negative $\rightarrow$ 0

Effect of Noise

- Noise can flip bit decisions at low SNR
- BPSK is more robust compared to ASK and FSK because phase is less affected by noise

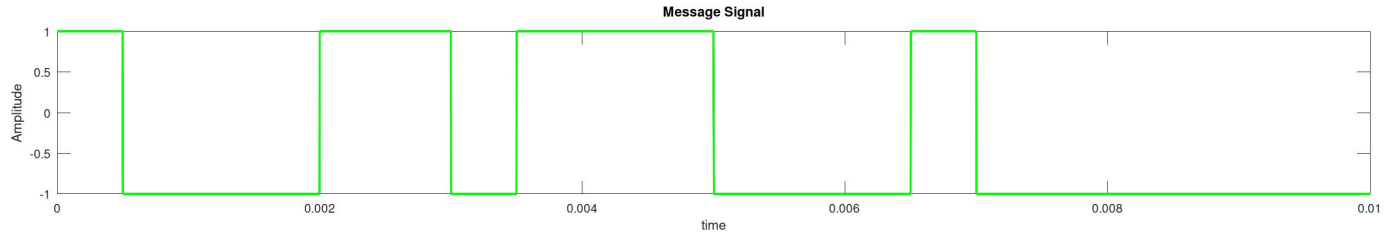
Code

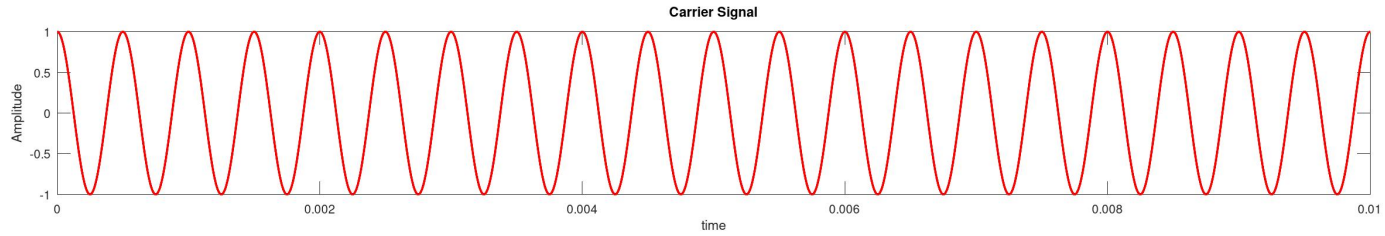
```
clc;
clear;

pkg load signal

tend = 0.01;
fs = 50000*10;
t = 0:1/fs:tend;
N = length(t);
dt = 1/fs;


fc = 2000;
c = cos(2*pi*fc*t)

n = 20;
```

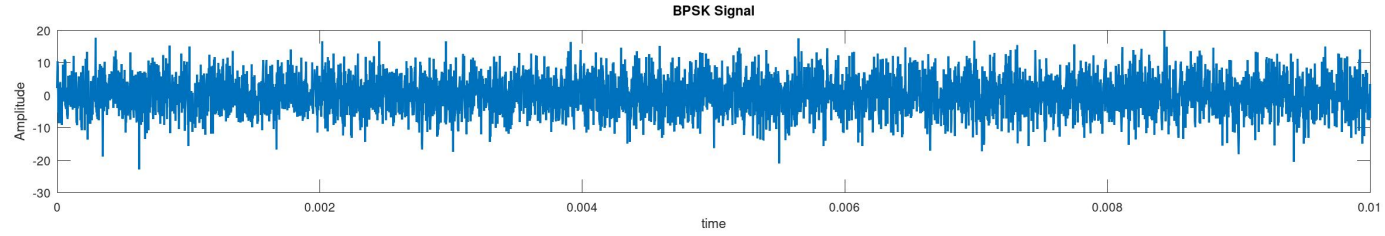
```
mn=2*randi([0 1],1,n)-1;
mt= zeros(1,N); #message signal
tb = N/n;
fm = 1/tb;
tbs = 0:tend/n:tend
s = tb/dt; #number of samples

##tb_index = (1/n)*[0:N-1]
##mt(1:100) = 1;

##a = mn(1)
##j = 1;
##for i = 1:N;
##  if (t(i)>tbs(j) && t(i)<tbs(j+1))
##    mt(i) = mn(j);
##    j = j+1;
##  end
##end


for i = 1:n;
  mt((i-1)*tb+1:i*tb) = mn(i);
end

bpsk = mt.*c;



noisy_bpsk = bpsk + sqrt(30)*randn(1,N);

set(0,"defaultlinelinewidth",2);
figure();
subplot(3,1,1);
plot(t,mt,'g');
title("Message Signal");
xlabel("time");
ylabel("Amplitude");

subplot(3,1,2);
plot(t,c,'r');
title("Carrier Signal");
xlabel("time");
ylabel("Amplitude");

subplot(3,1,3);
plot(t,noisy_bpsk);
title("BPSK Signal");
xlabel("time");
ylabel("Amplitude");
```

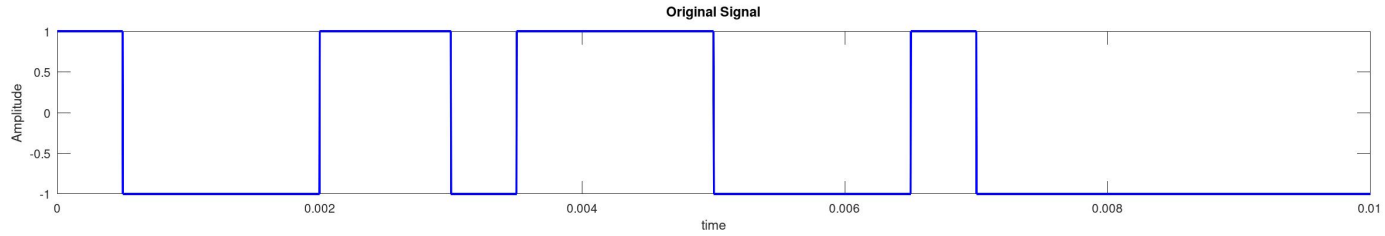
```
66
67
68
69
70  ##Demodulation
71
72  figure();
73
74  dm_signal1 = noisy_bpsk.*c;
75
76  new = zeros(1,N);
77
78  ##for i = 1:n;
79  ##  dm_signal2((i-1)*tb+1:i*tb) = mean(dm_signal1((i-1)*tb+1:i*tb));
80  ##end
81
82  for i = 1:n;
83    if(mean(dm_signal1((i-1)*tb+1:i*tb))>=0)
84      dm_signal2((i-1)*tb+1:i*tb)=1;
85    else
86      dm_signal2((i-1)*tb+1:i*tb)=-1;
87      end
88
89  end
90
91  subplot(3,1,1);
92  plot(t,mt,'b');
93  title("Original Signal");
94  xlabel("time");
95  ylabel("Amplitude");
96
97  subplot(3,1,2);
98  plot(t,dm_signal2,'m');
99  title("Demodulated Signal");
100 xlabel("time");
101 ylabel("Amplitude");
102
103
```

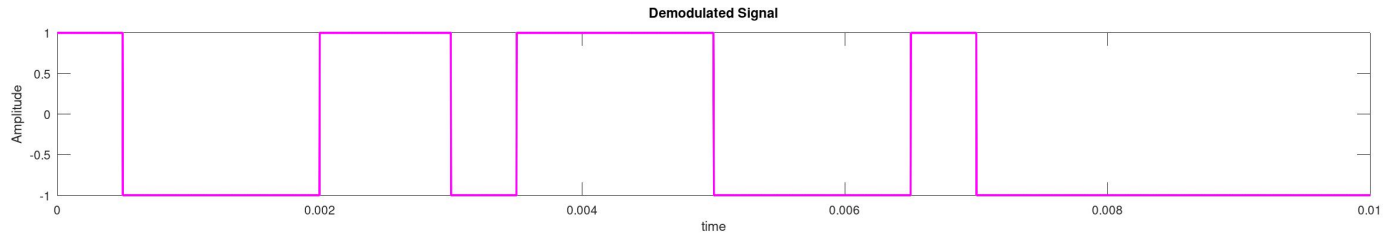
Simulation Results and Plots











Conclusion

The BPSK modulation and demodulation were successfully implemented using coherent detection. The original bit stream was accurately recovered, validating the system design. With noise, BPSK showed strong immunity compared to ASK. This experiment demonstrates BPSK's reliability and efficiency in digital communication systems under noisy channel conditions.

Experiment 12

Aim

Write a program to implement the Hamming Code error correction technique. For a given randomly generated bit sequence, display the transmitted codeword. Randomly flip a particular bit in the codeword and correct the error at the receiver side. Display the corrected codeword and the location of the error.

Software Required

Octave (version 9.4.0)

Theory

Hamming Code

Hamming Code is an error detection and correction technique used in digital communication to detect and correct single-bit errors.

Working Principle

1. Redundant Bits Addition Extra parity bits are added to the data bits. Number of parity bits r is chosen such that:

$$2^r \geq m + r + 1$$

   where m = number of data bits

2. Positioning of Parity Bits Parity bits are placed at positions that are powers of 2: (1, 2, 4, 8, . . .)
3. Parity Calculation Each parity bit checks specific positions in the codeword to ensure even/odd parity.

Error Detection & Correction

- At the receiver, parity bits are recalculated
- A syndrome is formed from parity checks
- Syndrome value gives the position of the error bit
- The erroneous bit is flipped to correct the codeword

Code

```
clc;
clear;


m = randi([0 1],1,8);
disp("Original Data:");
disp(m);

k = length(m);


r = 0;
while (2^r < (k + r + 1))
```

```
    r = r + 1;
end

n = k + r;

disp(["Number of parity bits: ", num2str(r)]);
disp(["Total codeword length: ", num2str(n)]);


pos = 2.^(0:r-1);


tx = zeros(1,n);

j = 1;
for i = 1:n
    if ~ismember(i, pos)
        tx(i) = m(j);
        j = j + 1;
    end
end

disp("After placing data bits:");
disp(tx);


for p = pos
    temp = 0;
    for i = 1:n
        if bitand(i, p) != 0
            temp = xor(temp, tx(i));
        end
    end
    tx(p) = temp;
end

disp("Encoded Codeword:");
disp(tx);

err_pos = randi([1 n]);
tx(err_pos) = 1 - tx(err_pos);

disp(["Error introduced at position: ", num2str(err_pos)]);
disp("Received Codeword:");
disp(tx);

decoded = tx(~ismember(1:n, pos));

disp("Decoded Data:");
```

```
63  disp(decoded);
```

Simulation Results and Plots

Output:

```
Original Data:
   0   1   1   1   0   0   1   0
Number of parity bits: 4
Total codeword length: 12
After placing data bits:
   0   0   0   0   1   1   1   0   0   0   1   0
Encoded Codeword:
   1   1   0   1   1   1   1   1   0   0   1   0
Error introduced at position: 5
Received Codeword:
   1   1   0   1   0   1   1   1   0   0   1   0
Error corrected at position: 5
Corrected Codeword:
   1   1   0   1   1   1   1   1   0   0   1   0
Decoded Data:
   0   1   1   1   0   0   1   0
```

Conclusion

The Hamming Code algorithm was successfully implemented to detect and correct single-bit errors. The transmitted data was encoded, an error was introduced, and the receiver accurately identified and corrected it. This experiment demonstrates the effectiveness of Hamming Code in improving data reliability in digital communication systems.

Experiment 13

Aim

Write a program to implement Huffman source encoding technique. Given a vector of probabilities for each alphabet, apply the Huffman source coding technique and output the bit sequence encodings for each alphabet.

Software Required

Octave (version 9.4.0)

Theory

Huffman Source Coding

Huffman coding is a lossless data compression technique used to assign variable-length binary codes to symbols based on their probabilities.
Working Principle
Symbols with higher probability → shorter codes
Symbols with lower probability → longer codes

This reduces the average code length and improves efficiency.

Steps in Huffman Coding

- Arrange symbols in ascending order of probability
- Combine two lowest-probability symbols to form a new node
- Repeat until a single tree (Huffman tree) is formed
- Assign: Left branch → 0 Right branch → 1
- Traverse the tree to get binary codes for each symbol

Properties

- Produces prefix codes (no code is a prefix of another)
- Ensures unique decodability
- Minimizes average code length

Code

```
clc;
clear;

symbols = ['A', 'B', 'C', 'D'];
p = [0.4, 0.3, 0.2, 0.1];

n = length(p);

nodes = cell(n,1);
for i = 1:n
    nodes{i} = struct('sym', symbols(i), 'prob', p(i), ...
                      'left', [], 'right', []);
end
```

```

while length(nodes) > 1
    probs = cellfun(@(x) x.prob, nodes);
    [~, idx] = sort(probs);
    nodes = nodes(idx);

    left = nodes{1};
    right = nodes{2};

    newNode = struct('sym', '', ...
                     'prob', left.prob + right.prob, ...
                     'left', left, ...
                     'right', right);

    nodes = nodes(3:end);
    nodes{end+1} = newNode;
end

root = nodes{1};

function generateCodes(node, code)
    if isempty(node.left) && isempty(node.right)
        fprintf('%s : %s\n', node.sym, code);
        return;
    end

    if ~isempty(node.left)
        generateCodes(node.left, strcat(code, '0'));
    end

    if ~isempty(node.right)
        generateCodes(node.right, strcat(code, '1'));
    end
end

disp('Huffman Codes:');
generateCodes(root, '');
```

Simulation Results and Plots

Output:

```
Huffman Codes:
A : 0
B : 10
D : 110
C : 111
```

Conclusion

Huffman coding is an efficient lossless compression technique that reduces data size by assigning variable-length codes based on symbol probabilities. It ensures optimal encoding with minimum average code length. The use of prefix codes guarantees unique and error-free decoding. This method is widely used in data compression applications. Overall, it improves storage and transmission efficiency without losing information.

Experiment 14

Aim

Write a program to implement Quadrature Phase Shift Keying (QPSK). Generate two random bit sequence, one for the I-stream and one for the Q-stream and modulate them with cosine and sine carrier waveforms respectively. Add noise and demodulate the signal using a coherent detector using both the cosine and sine carriers at the receiver as well. Plot the reconstructed bit stream.

Software Required

Octave (version 9.4.0)

Theory

QPSK

Quadrature Phase Shift Keying (QPSK) is a digital modulation technique in which two bits are transmitted simultaneously by changing the phase of a carrier signal.

The input bit stream is split into two parts: I (In-phase) stream Q (Quadrature) stream These are modulated using two orthogonal carriers: I → cosine carrier Q → sine carrier The combined signal has four possible phase states (00, 01, 10, 11)

Modulated Signal

$s(t) = I(t)cos(2\pi f_c t) + Q(t)sin(2\pi f_c t)$

Demodulation (Coherent Detection)

Received signal is multiplied with: cosine carrier → recover I-stream and sine carrier → recover Q-stream Signals are integrated and thresholded to reconstruct bits

Effect of Noise

- Noise affects both amplitude and phase
- Errors may occur at low SNR
- QPSK is more bandwidth-efficient and performs better than BPSK for higher data rates

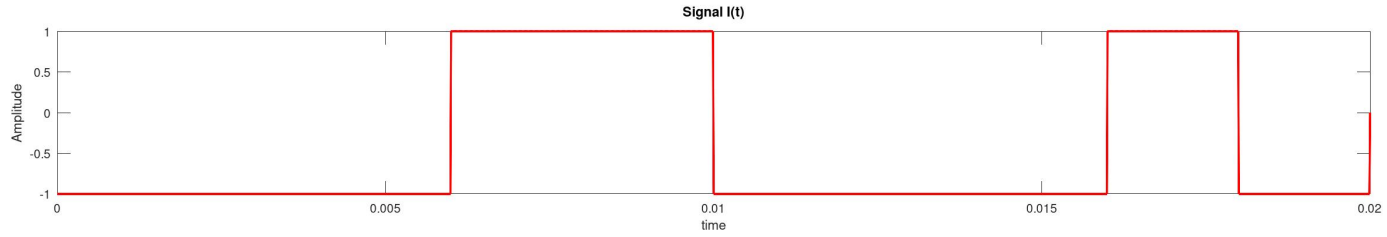
Code

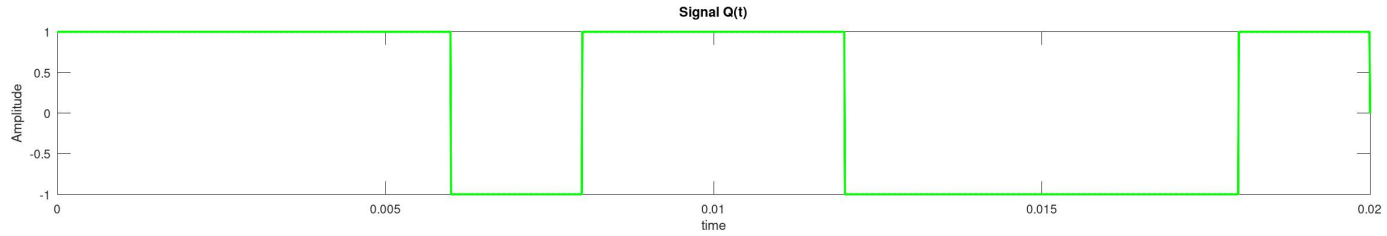
```
clc;
clear;

fs = 1000*100;  ##Sampling Frequency
Ts = 1/fs; ##Sampling Period
Tend = 0.02;
t = 0:Ts:Tend;
N = length(t); ##Total Samples

numOfBits = 10;
bitPeriod = Tend/numOfBits;
samplesPerBit = bitPeriod/Ts;
in = randi([0 1],1,numOfBits);
qn = randi([0 1],1,numOfBits);
```

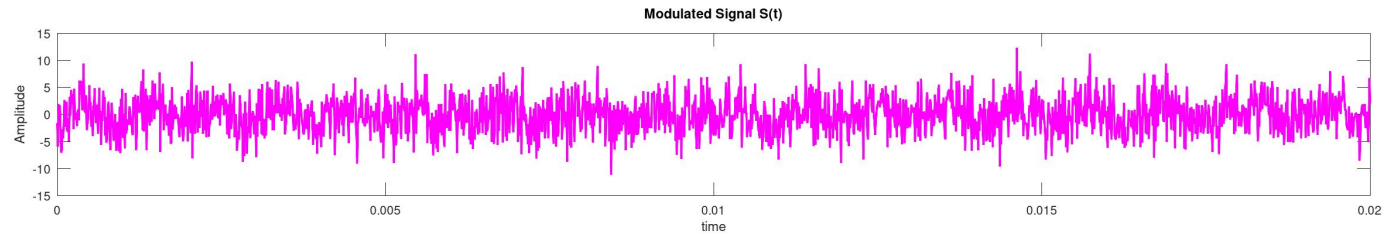
```
15
16  I = zeros(1,N);
17  Q = zeros(1,N);
18
19  for i = 1:numOfBits
20    if(in(i)==0)
21      in(i) = -1;
22    endif
23
24    if(qn(i)==0)
25      qn(i) = -1;
26    endif
27  endfor
28
29  for i = 1:numOfBits
30    I((i-1)*samplesPerBit+1:i*samplesPerBit) = in(i);
31    Q((i-1)*samplesPerBit+1:i*samplesPerBit) = qn(i);
32  endfor
33
34
35  fc = 1000; ##Carrier Frequency
36  S = I.*cos(2*pi*fc*t) + Q.*sin(2*pi*fc*t) + sqrt(10)*randn(1,N);
37
38
39  set(0,"defaultlinelinewidth",2);
40
41  subplot(3,1,1);
42  plot(t,I,'r');
43  title("Signal I(t)");
44  xlabel("time");
45  ylabel("Amplitude");
46
47  subplot(3,1,2);
48  plot(t,Q,'g');
49  title("Signal Q(t)");
50  xlabel("time");
51  ylabel("Amplitude");
52
53  subplot(3,1,3);
54  plot(t,S,'m');
55  title("Modulated Signal S(t)");
56  xlabel("time");
57  ylabel("Amplitude");
58
59
60  ##Receiver
61
62  DMI = S.*cos(2*pi*fc*t);
63  DMQ = S.*sin(2*pi*fc*t);
```

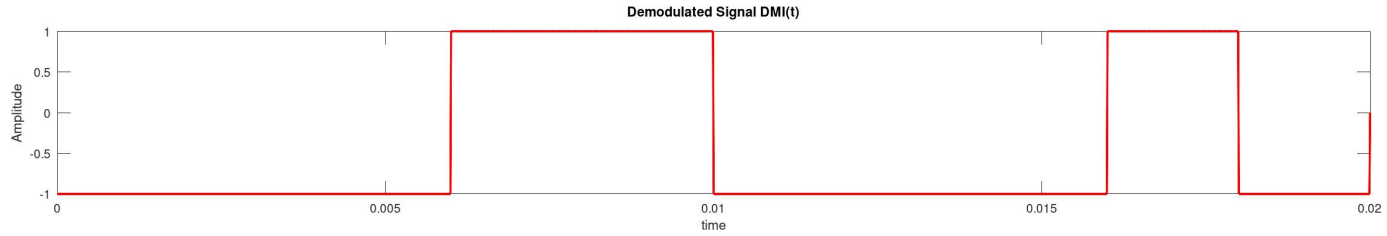
```
64
65  for i = 1:numOfBits
66    DMI((i-1)*samplesPerBit+1:i*samplesPerBit) =
      ↪  sum(DMI((i-1)*samplesPerBit+1:i*samplesPerBit));
67    DMQ((i-1)*samplesPerBit+1:i*samplesPerBit) =
      ↪  sum(DMQ((i-1)*samplesPerBit+1:i*samplesPerBit));
68  endfor
69
70  ##Comparator
71  for i = 1:numOfBits
72    if(mean(DMI((i-1)*samplesPerBit+1:i*samplesPerBit))>=0)
73      DMI((i-1)*samplesPerBit+1:i*samplesPerBit) = 1;
74    else
75      DMI((i-1)*samplesPerBit+1:i*samplesPerBit) = -1;
76    endif
77
78  if(mean(DMQ((i-1)*samplesPerBit+1:i*samplesPerBit))>=0)
79      DMQ((i-1)*samplesPerBit+1:i*samplesPerBit) = 1;
80    else
81      DMQ((i-1)*samplesPerBit+1:i*samplesPerBit) = -1;
82    endif
83  endfor
84  DMI(N)=0;
85  figure();
86
87  subplot(3,1,1);
88  plot(t,DMI,'r');
89  title("Demodulated Signal DMI(t)");
90  xlabel("time");
91  ylabel("Amplitude");
92
93  subplot(3,1,2);
94  plot(t,DMQ,'g');
95  title("Demodulated Signal DMQ(t)");
96  xlabel("time");
97  ylabel("Amplitude");
98
99
100
101
```

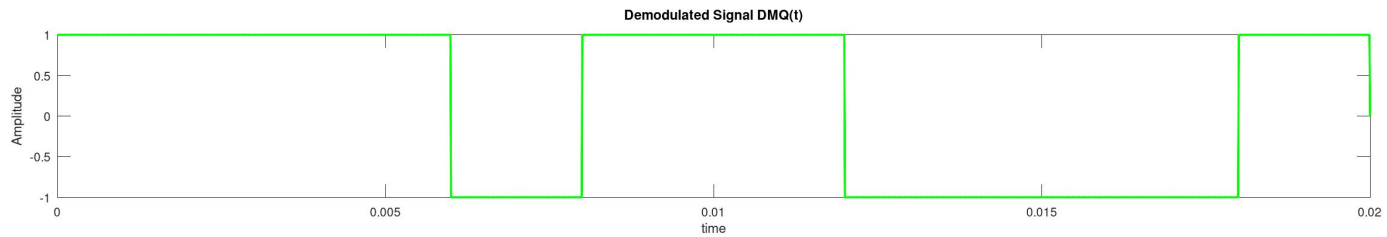
Simulation Results and Plots











Conclusion

QPSK is an efficient digital modulation technique that transmits two bits per symbol, improving bandwidth utilization. It uses phase variations of a carrier to encode data, making it more efficient than BPSK. With coherent detection, accurate recovery of I and Q streams is possible. Although noise can introduce errors at low SNR, QPSK still offers good performance. Overall, it provides a balance between data rate and reliability.

Experiment 15

Aim

Write a program to plot a Bit Error Rate (BER) vs Signal to Noise Ratio (SNR) graph. Using ASK and BPSK modulation techniques, plot a graph where the x-axis is SNR in dB, while y- axis is BER. Use the semilogy() function and take appropriate noise and signal power values to plot the graph.

Software Required

Octave (version 9.4.0)

Theory

BER vs SNR

Bit Error Rate (BER) is the probability of a bit being received incorrectly in a communication system. It is a key measure of system performance.

Signal-to-Noise Ratio (SNR)

- SNR represents the strength of the signal relative to noise
- Usually expressed in decibels (dB)
- Higher SNR → better signal quality → lower BER

BER in Modulation Techniques

- ASK (Amplitude Shift Keying): BER is higher because noise directly affects amplitude
- BPSK (Binary Phase Shift Keying): BER is lower as it depends on phase, which is less sensitive to noise

Code

- ASK

```
clc;
clear;

pkg load signal

tend = 0.01;
fs = 500000*10;
t = 0:1/fs:tend;
N = length(t);
dt = 1/fs;


fc = 5000;
c = cos(2*pi*fc*t);


n = 2000;
mn = randi([0 1],1,n);

```

```
20
21  tb = tend/n;
22  s = floor(tb/dt);
23
24
25  mt = zeros(1,N);
26  for i = 1:n
27      mt((i-1)*s+1 : min(i*s,N)) = mn(i);
28  end
29
30
31  ask = mt .* c;
32
33  SignalPower = mean(ask.^2);
34
35  noise = [0.1 0.5 1 5 10 50 100 500];
36  l_noise = length(noise);
37
38  SNR = zeros(1,l_noise);
39  BER = zeros(1,l_noise);
40
41
42  for j = 1:l_noise
43
44      No = noise(j);
45      Noise = sqrt(No)*randn(1,N);
46
47      noisy_ask = ask + Noise;
48
49      NoisePower = mean(Noise.^2);
50      SNR(j) = 10*log10(SignalPower/NoisePower);
51
52
53      a = abs(noisy_ask);
54
55
56      fm = 1/tb;
57      coeff = fir1(200, (fm)/(fs/2));
58      fil = filter(coeff,1,a);
59
60
61      rx_bits = zeros(1,n);
62
63      for i = 1:n
64          segment = fil((i-1)*s+1 : min(i*s,N));
65
66          if mean(segment) > mean(fil)/2
67              rx_bits(i) = 1;
68          else
```

```
            rx_bits(i) = 0;
        end
    end

    numOfError = sum(rx_bits != mn);
    BER(j) = numOfError / n;
end


figure();
semilogy(SNR, BER, 'o-');
grid on;
title("BER vs SNR (ASK)");
xlabel("SNR (dB)");
ylabel("BER");


```

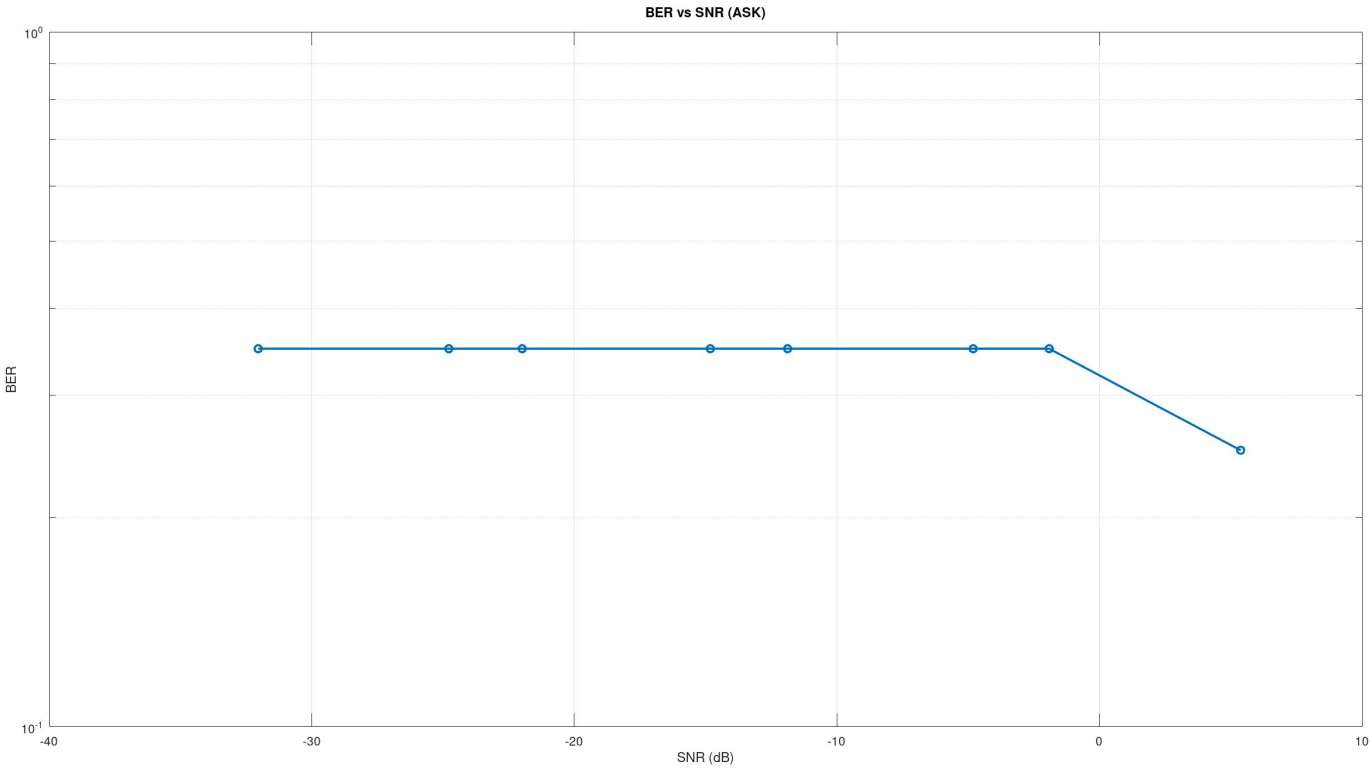
- BPSK

```
clc;
clear;

pkg load signal

tend = 0.01;
fs = 50000*10;
t = 0:1/fs:tend;
N = length(t);
dt = 1/fs;

fc = 2000;
c = cos(2*pi*fc*t);

n = 2000;
mn = 2*randi([0 1],1,n)-1;

tb = floor(N/n);
mt = zeros(1,N);

for i = 1:n
  mt((i-1)*tb+1 : min(i*tb,N)) = mn(i);
end

bpsk = mt .* c;
SignalPower = mean(bpsk.^2);

noise = [0.1 0.5 1 5 10 20 50 100];
l_noise = length(noise);
```

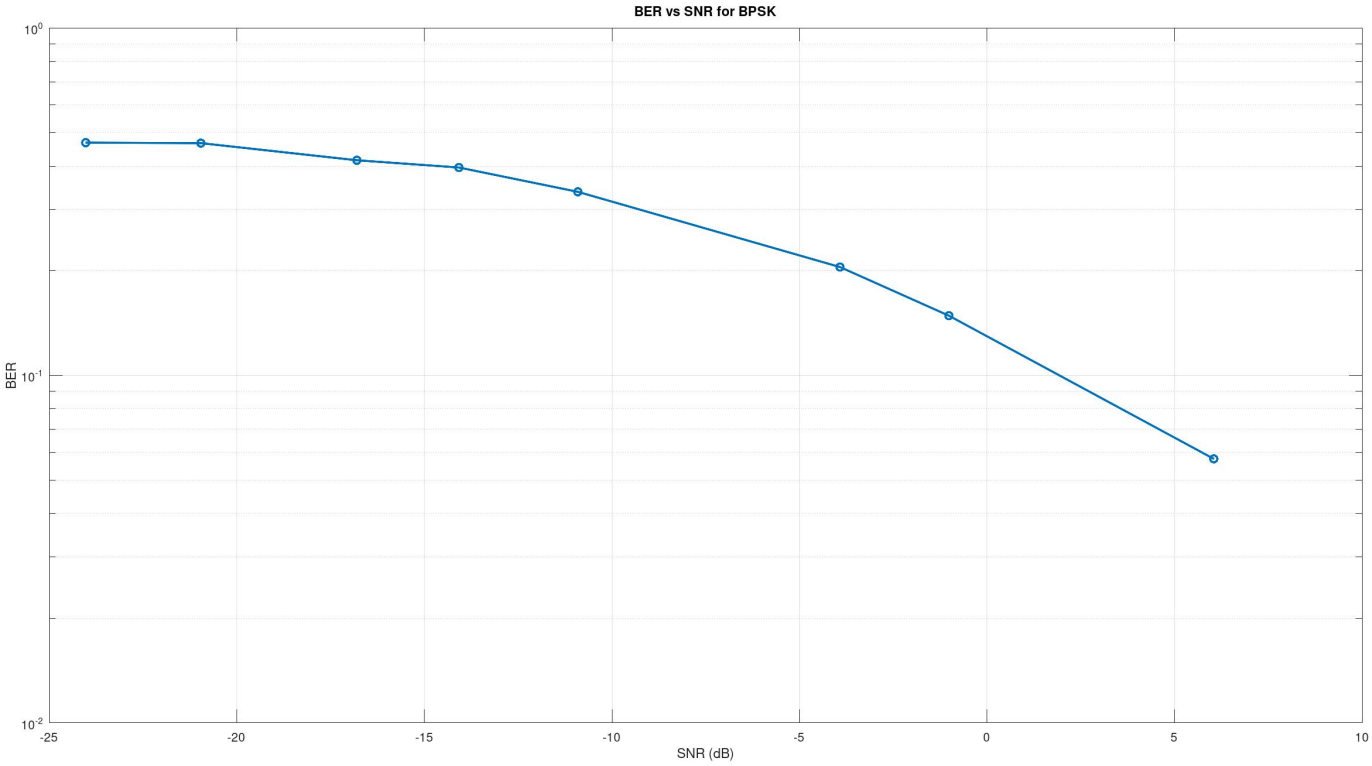
```
30
31  SNR = zeros(1,l_noise);
32  BER = zeros(1,l_noise);
33
34  for j = 1:l_noise
35
36    No = noise(j);
37    Noise = sqrt(No)*randn(1,N);
38    noisy_bpsk = bpsk + Noise;
39
40    NoisePower = mean(Noise.^2);
41    SNR(j) = 10*log10(SignalPower/NoisePower);
42
43    dm_signal1 = noisy_bpsk .* c;
44
45    rx_bits = zeros(1,n);
46
47    for i = 1:n
48      segment = dm_signal1((i-1)*tb+1 : min(i*tb,N));
49      if mean(segment) >= 0
50        rx_bits(i) = 1;
51      else
52        rx_bits(i) = -1;
53      end
54    end
55
56    numOfError = sum(rx_bits != mn);
57    BER(j) = numOfError / n;
58
59  end
60
61  figure();
62  semilogy(SNR, BER, 'o-');
63  grid on;
64  xlabel("SNR (dB)");
65  ylabel("BER");
66  title("BER vs SNR for BPSK");
```

Simulation Results and Plots

ASK



BPSK



Conclusion

The BER vs SNR graph illustrates how error performance improves as signal quality increases. As SNR increases, BER decreases rapidly, showing more reliable communication. ASK exhibits higher error rates due to its sensitivity to noise, while BPSK performs better with lower BER. The use of a logarithmic scale helps visualize this variation clearly. Overall, BPSK is more robust and suitable for noisy environments.